SOFTWARE ENGINEERING

Module 1 + Module 2 Complete University Notes

Beginner-friendly • Exam-friendly • Diagram-rich

Based primarily on Roger S. Pressman & Bruce R. Maxim,
Software Engineering: A Practitioner's Approach, 9th Edition (2019)

```
MODULE 1                    MODULE 2
Software & SE               Process Models
Nature of Software          Waterfall / Prototype
Application Domains         Evolutionary / Spiral
Legacy Software             Unified Process
Process Framework           Agile / Scrum / XP
                            Kanban / DevOps
```

Contents

PART I — MODULE 1: Introduction to Software Engineering

• 1. What is Software?

• 2. What is Software Engineering?

• 3. Why Software Engineering is Needed

• 4. Software Engineering as a Layered Technology

• 5. Nature of Software

• 6. Software Application Domains

• 7. Legacy Software

• 8. Software Process

• 9. Process Flow Types

• 10. Umbrella Activities

PART II — MODULE 2: Process Models and Agile Development

• 1. Software Process Models

• 2. Waterfall Model

• 3. Prototyping Model

• 4. Incremental Model

• 5. Evolutionary Process Model

• 6. Spiral Model

• 7. Concurrent Development Model

• 8. Unified Process

• 9. Agility

• 10. Cost of Change

• 11. Agile Process and Values

• 12. Scrum

• 13. Extreme Programming (XP)

• 14. Kanban

• 15. DevOps

• 16. Iterative vs Incremental

• 17. Model Comparison

• 18. Important Exam Diagrams

• 19. High-Priority Exam Questions

MODULE 1

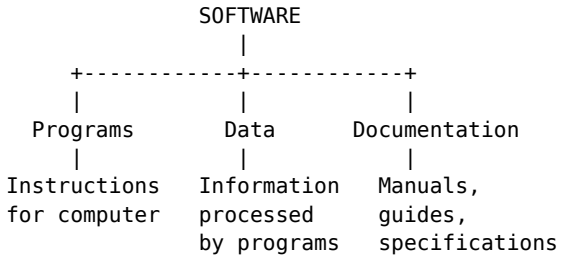
Introduction to Software Engineering, Nature of Software, Software Application Domains, Legacy Software

Module focus

Understand what software is, why engineering principles are necessary, how software differs from hardware, where software is used, why legacy systems matter, and how a generic software process works.

1. What is Software?

Software is a collection of programs, data, and related information that enables a computer system to perform useful tasks. It is not just source code; the complete software product also includes the information needed to use, operate, and maintain it.



Example: a banking system contains programs for login and transactions, customer/account data, and user or technical documentation.

2. What is Software Engineering?

Software engineering is the systematic and disciplined application of engineering principles, methods, and tools to the development, operation, and maintenance of software. In simple terms, it is an organized way of building software so that it is useful, reliable, secure, maintainable, and economical.

```
Problem → Requirements → Planning → Analysis/Modeling
      → Design → Coding → Testing → Deployment → Maintenance/Evolution
```

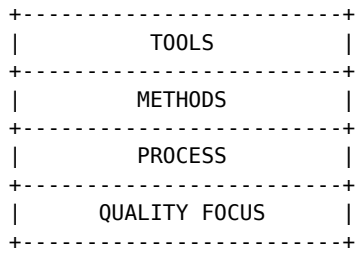
3. Why is Software Engineering Needed?

Software systems can be large, complex, safety-critical, and costly. Coding without a controlled process can lead to misunderstood requirements, defects, cost overruns, schedule delays, security problems, and difficult maintenance.

- clarify requirements
- manage complexity
- estimate time, effort, cost, and resources
- reduce defects and project risk
- improve software quality
- support maintenance and evolution
- coordinate teams and stakeholders

4. Software Engineering as a Layered Technology

Pressman presents software engineering as a layered technology. Quality focus is the foundation; the process organizes the work; methods provide the technical how-to; tools provide automated or semi-automated support.



Easy memory: Quality → Process → Methods → Tools.

5. Nature of Software

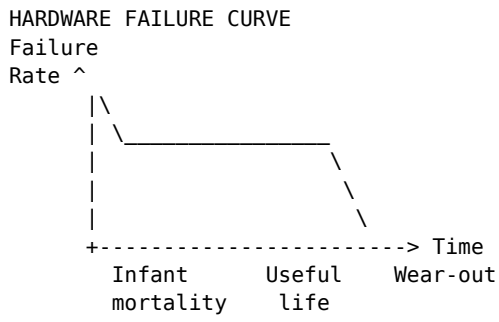
5.1 Developed, not manufactured

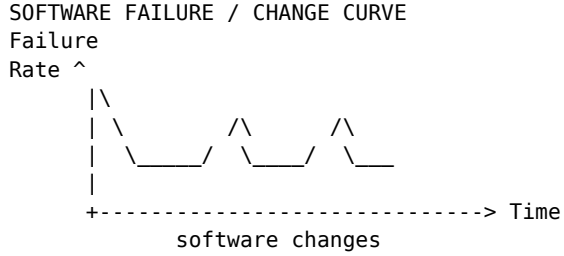
Hardware is physically manufactured, while software is mainly created through intellectual activities such as analysis, design, coding, and testing. After the first copy is developed, making additional copies is normally inexpensive compared with the original development effort.

```
Requirements → Analysis → Design → Coding → Testing → Software Product
```

5.2 Software does not wear out like hardware

Software does not physically wear out. However, it can deteriorate because changes can introduce faults, side effects, and increasing complexity.





Exam point

Hardware has physical wear-out; software does not physically wear out, but maintenance changes can introduce new faults and increase complexity.

5.3 Software is intangible

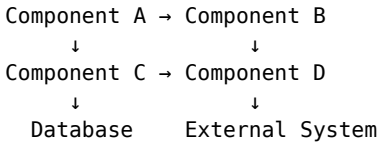
Software is an intellectual product. Algorithms, program logic, data structures, and behaviour cannot be physically touched like hardware.

5.4 Software is highly changeable

Requirements, business rules, technology, regulations, and security threats can change. Therefore, software commonly evolves after its first release.

5.5 Software complexity

Large systems contain many interacting components. More components and dependencies make development, testing, debugging, and maintenance harder. Architecture, modularity, abstraction, and testing help control this complexity.

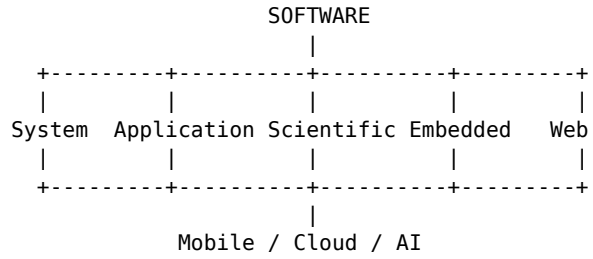


5.6 Software is often custom-built

Many systems are created for a specific organization or purpose, such as hospital, banking, or university systems. Reusable libraries and frameworks reduce new work, but complete systems often still need customization.

6. Software Application Domains

A software application domain is an area in which software is developed and used. Different domains impose different requirements, risks, and constraints.



6.1 System Software

Manages computer resources and provides services for application software. Examples: operating systems, compilers, device drivers, and utilities.

6.2 Application Software

Performs tasks directly useful to users, such as banking, accounting, word processing, and hospital management.

6.3 Engineering and Scientific Software

Supports computation, analysis, simulation, and modeling, such as weather simulation and engineering analysis.

6.4 Embedded Software

Runs inside a hardware product and controls or monitors its functions, such as software in cars, washing machines, medical devices, and industrial controllers.

6.5 Product-Line Software

A family of related products sharing common architecture and components.

6.6 Web Applications

Applications accessed through web technologies. Key concerns include security, usability, performance, availability, and scalability.

6.7 Mobile Applications

Apps for phones and tablets. They must consider touch interfaces, battery use, connectivity, screen sizes, and limited resources.

6.8 Cloud-Based Applications

Software delivered using cloud infrastructure and services. Key concerns include scalability, availability, security, and performance.

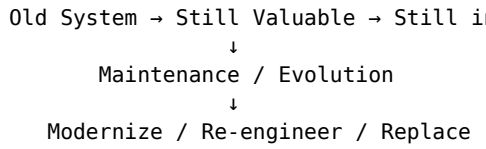
6.9 Artificial Intelligence Software

Software that performs intelligent processing, such as machine learning, speech recognition, computer vision, recommendation systems, and generative AI.

Domain	Examples	Key concern
System	OS, compiler, driver	Resource management
Application	Banking, accounting	User/business needs
Scientific/Engineering	Simulation, analysis	Accuracy/performance
Embedded	Car, medical device	Timing/reliability/safety
Web	Banking portal, e-commerce	Security/scalability
Mobile	Android/iOS apps	Battery/usability/connectivity
Cloud	SaaS, cloud storage	Availability/scalability
AI	ML, vision, speech	Data/model quality

7. Legacy Software

Legacy software is an existing system that continues to provide important value even though it may use old technology, old design, or difficult-to-maintain code. A system is “legacy” mainly because it remains valuable while replacement or major modification is costly or risky.



7.1 Why legacy software remains

- contains valuable business rules
- contains important historical data
- provides proven functionality
- integrates with other systems
- contains years of organizational knowledge

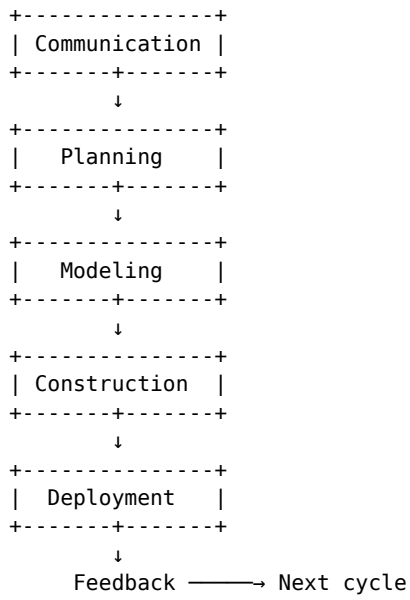
7.2 Problems of legacy software

- outdated languages, platforms, databases, or hardware
- poor or incomplete documentation
- difficult maintenance
- integration difficulties with modern systems

- security limitations
- high maintenance cost and dependence on specialized knowledge

8. Software Process

A software process is a structured set of activities used to develop and maintain software. Pressman's generic framework can be viewed as communication, planning, modeling, construction, and deployment, with feedback causing further cycles.



Communication

Understand customer needs, stakeholders, and requirements.

Planning

Estimate effort, resources, time, cost, schedule, and risks.

Modeling

Create analysis and design representations of the system.

Construction

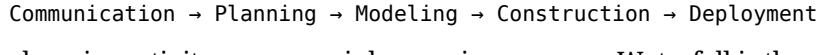
Implement code and perform testing.

Deployment

Deliver the software, obtain feedback, and support continued improvement.

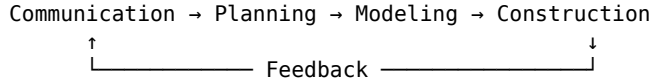
9. Process Flow Types

9.1 Linear flow



Each major activity occurs mainly once in sequence. Waterfall is the classic example.

9.2 Iterative flow



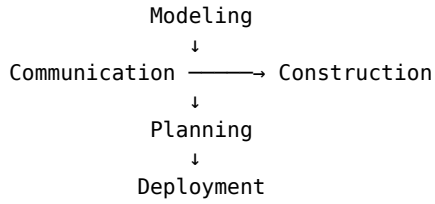
The process repeats so the solution can be refined.

9.3 Evolutionary flow

```
Cycle 1 → Increment 1
Cycle 2 → Increment 2
Cycle 3 → Increment 3
Cycle 4 → Increment 4
```

Successive releases make the system more complete.

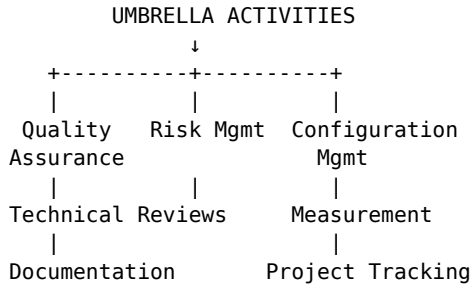
9.4 Parallel flow



Activities can overlap rather than wait for every other activity to finish.

10. Umbrella Activities

Umbrella activities occur throughout the software process and support the primary development activities.



- software quality assurance
- software configuration management
- technical reviews
- risk management
- measurement
- project tracking
- documentation
- reuse management

MODULE 2

Process Models, Agility and Agile Processes, Scrum, XP, Kanban, DevOps

Module focus

Understand major software process models, why adaptive methods emerged, and how Scrum, XP, Kanban, and DevOps organize iterative development and continuous delivery.

1. Software Process Models

A software process model is a simplified representation of how software development activities are organized. The suitable model depends on requirements, project size, risk, team characteristics, and how much change is expected.

```
Process Models
   |
   +-- Waterfall
   +-- Prototyping
   +-- Evolutionary
   |      +-- Incremental
   |      +-- Spiral
   |      +-- Concurrent
   +-- Unified Process
   +-- Agile approaches
```

2. Waterfall Model

The Waterfall model is a largely sequential process in which work flows through a set of ordered phases.

```
Requirements Definition
          ↓
System & Software Design
          ↓
Implementation & Unit Testing
          ↓
Integration & System Testing
          ↓
Operation & Maintenance
```

Advantages

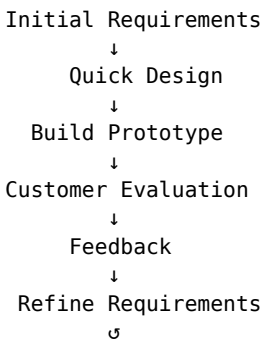
- simple and easy to understand
- clear milestones and deliverables
- easy to manage and document
- useful when requirements are stable

Disadvantages

- change is difficult or expensive
- customer feedback may come late
- working software appears relatively late
- late discovery of problems can be costly
- poor fit for uncertain or rapidly changing projects

3. Prototyping Model

A prototype is an early version of a system used to clarify requirements or explore a solution. It is especially useful when stakeholders cannot describe exactly what they need.



Advantages

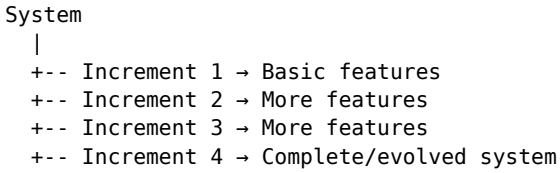
- clarifies requirements
- provides early user feedback
- reduces misunderstanding
- useful when requirements are uncertain

Disadvantages

- prototype may be mistaken for final software
- quick design choices may create poor architecture
- users may expect unrealistic delivery speed

4. Incremental Process Model

In incremental development, the system is delivered through several releases, with each release adding useful functionality.

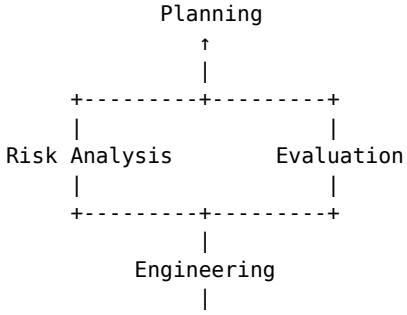


5. Evolutionary Process Model

Evolutionary development builds a system through repeated versions. Feedback from one version is used to guide the next.

6. Spiral Process Model

The Spiral model combines iterative development with explicit risk analysis. Each cycle identifies objectives, analyzes risks, performs engineering work, evaluates the result, and plans the next cycle.



```
                ↓
          Next Spiral
```

Exam point

Key strength of the Spiral model: explicit risk management. It is useful for large, complex, high-risk projects.

7. Concurrent Development Model

The concurrent model recognizes that different engineering activities can be in different states at the same time. One activity may be under development while another is under review or revision.

```
Activity states:
Inactive → Under Development → Under Review
                          ↓
                    Under Revision
                          ↓
                      Baselined
                          ↓
                           Done
```

8. Unified Process

The Unified Process (UP) is an iterative and incremental framework with four major phases. Each phase can contain multiple iterations.

```
Inception → Elaboration → Construction → Transition
    \         Iterations / Increments        /
```

Inception

Defines the business case, scope, major requirements, feasibility, and initial risks.

Elaboration

Refines requirements, establishes or validates architecture, and addresses major risks.

Construction

Implements and tests most of the system functionality.

Transition

Delivers the system to users and handles deployment, feedback, and release-related issues.

9. Agility

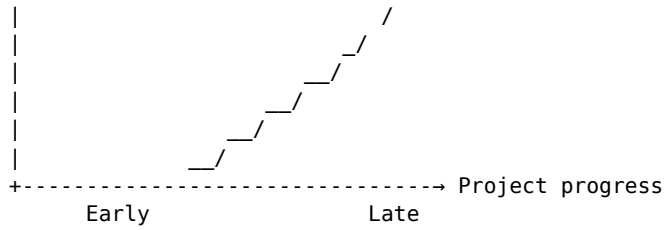
Agility is the ability of a software team to respond effectively to changing requirements, business priorities, technology, regulations, and customer expectations.

```
Change can come from:
Requirements • Business • Technology • Regulations • Market • Customers
                                ↓
                          AGILITY = adaptation
```

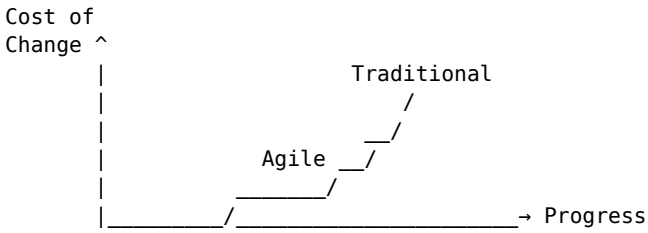
10. Cost of Change

A common software-engineering idea is that changing a system later in the lifecycle can be more expensive because more design, code, testing, integration, and documentation may already depend on earlier decisions.

```
Cost of
Change ^
       |                          Traditional
```

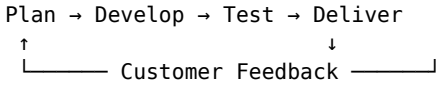


Agile engineering practices such as frequent feedback, automated testing, refactoring, continuous integration, and small releases attempt to make change more manageable throughout development.



11. Agile Process and Values

Agile development is generally iterative, incremental, feedback-driven, and adaptive. Teams build small pieces of useful functionality, obtain feedback, and use that feedback to guide the next cycle.



Agile Manifesto values

```
Individuals & interactions   >   processes & tools
Working software             >   comprehensive documentation
Customer collaboration       >   contract negotiation
Responding to change         >   following a plan
```

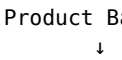
The right-hand items still have value; the Agile Manifesto emphasizes the left-hand items more strongly.

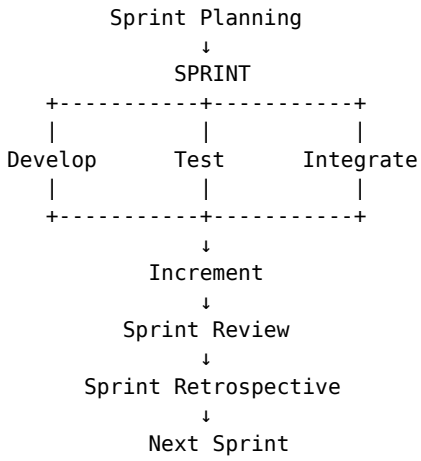
Important Agile principles

- deliver valuable software early and continuously
- welcome changing requirements
- deliver working software frequently
- maintain close customer/developer collaboration
- support motivated teams
- prefer effective direct communication where practical
- treat working software as an important measure of progress
- maintain a sustainable pace
- pursue technical excellence and good design
- keep solutions simple
- support self-organizing teams
- regularly reflect and improve the process

12. Scrum

Scrum is an agile framework for developing and managing complex products. Work is organized into fixed-length Sprints that produce a usable Increment.





12.1 Scrum accountabilities

Product Owner

Accountable for maximizing product value and effective Product Backlog management.

Scrum Master

Helps establish Scrum, supports the team, and helps address impediments to effective Scrum.

Developers

Create the usable product Increment during the Sprint.

12.2 Scrum artifacts

Product Backlog: ordered list of work needed or desired for the product.

Sprint Backlog: selected Product Backlog items plus the plan for achieving the Sprint Goal.

Increment: usable, integrated product outcome created during the Sprint.

12.3 Scrum events

Sprint Planning

Decides what the Sprint should accomplish and how the work will be approached.

Daily Scrum

A short daily event where Developers inspect progress toward the Sprint Goal and adapt the plan.

Sprint Review

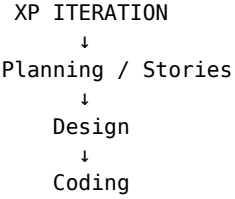
Stakeholders and the Scrum Team inspect the result and adapt future work.

Sprint Retrospective

The team reflects on how the Sprint went and identifies improvements.

13. Extreme Programming (XP)

Extreme Programming is an agile approach that emphasizes technical excellence, short feedback cycles, customer involvement, simple design, and disciplined engineering practices.



```
      ↓
   Testing
      ↓
  Release
     ↺
```

Important XP practices

Pair Programming

Two developers work together on the same task, improving review, knowledge sharing, and problem solving.

Test-Driven Development (TDD)

Write a test, see it fail, implement enough code to pass, then refactor. Remember: Red → Green → Refactor.

Continuous Integration

Integrate changes frequently into a shared codebase and run builds/tests to catch problems early.

Refactoring

Improve the internal structure of code without intentionally changing externally observable behaviour.

Small Releases

Deliver useful functionality frequently rather than waiting for one very large release.

User Stories

Describe desired functionality from the user's point of view, e.g., "As a student, I want to view my attendance..."

```
TDD:
Write Test → Test Fails → Write Code → Test Passes → Refactor → repeat
```

14. Kanban

Kanban is a workflow-management method that emphasizes visualizing work, limiting work in progress, managing flow, and continuous improvement.

```
+----------+----------+----------+----------+
|  TO DO   |  DOING   | TESTING  |   DONE   |
+----------+----------+----------+----------+
| Task A   | Task C   | Task E   | Task G   |
| Task B   | Task D   |          | Task H   |
+----------+----------+----------+----------+
```

Tasks move through the workflow from left to right. The exact columns can be adapted to the team's process.

14.1 WIP — Work in Progress

WIP is the amount of work currently being processed. Kanban commonly uses WIP limits to prevent excessive simultaneous work.

```
DOING — WIP limit = 2
+----------+
| Task A   |
| Task B   |
+----------+
New work waits until capacity is available.
```

Too much WIP causes context switching, bottlenecks, and longer lead time. Limiting WIP encourages finishing work before starting too much new work.

14.2 Important Kanban practices

- visualize workflow
- limit work in progress

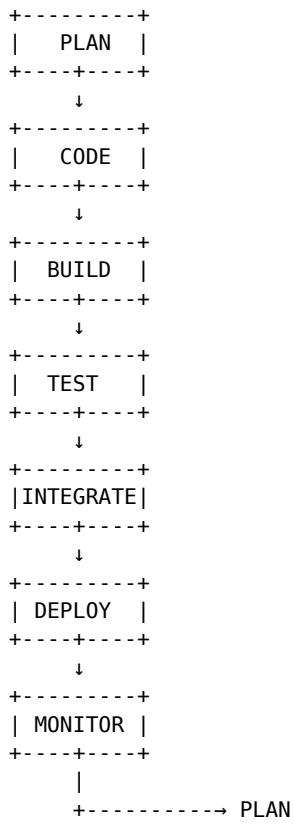
- manage flow
- make policies explicit
- use feedback loops
- improve continuously

15. DevOps

DevOps combines development and operations through collaboration, automation, continuous delivery, monitoring, and shared responsibility. The aim is to improve the speed, reliability, and repeatability of software delivery and operation.

```
Development
     ↕
Collaboration + Automation
     ↕
 Operations
```

15.1 DevOps lifecycle



15.2 Continuous Integration (CI)

Developers frequently integrate code into a shared repository. Automated build and testing then provide rapid feedback.

```
Code Change → Shared Repository → Build → Automated Tests → Feedback
```

15.3 Continuous Delivery

Software is kept in a state where it can be released reliably when needed.

```
Code → Build → Test → Package → Release-ready
```

15.4 Continuous Deployment

Changes that pass the required checks are automatically deployed to production.

```
Code → Build → Test → Automatic Deployment → Production
```

15.5 Continuous Monitoring

The running system is continuously observed for performance, failures, availability, and operational signals. Monitoring data feeds back into development and improvement.

```
Production → Monitoring → Metrics/Problems → Feedback → Development
```

16. Iterative vs Incremental

Concept	Meaning	Example
Iterative	Repeatedly refine and improve the solution	v1 → improve → v2 → improve → v3
Incremental	Add useful functionality piece by piece	Login → Login+Profile → Login+Profile+Payment

Agile development commonly combines both: repeated iterations produce a growing set of increments.

```
Iteration 1 → Increment 1
Iteration 2 → Increment 2
Iteration 3 → Increment 3
```

17. Traditional vs Agile

Aspect	Traditional / plan-driven	Agile / adaptive
Requirements	Often defined early	Can evolve
Flow	Sequential phases may dominate	Iterative/incremental
Feedback	Often later	Frequent
Change	May be expensive	Expected and managed
Delivery	Working software often later	Frequent working increments

18. Scrum vs XP vs Kanban vs DevOps

Aspect	Scrum	XP	Kanban	DevOps
Main focus	Product development framework	Engineering practices	Workflow / flow	Delivery + operations
Work style	Sprints	Short iterations/releases	Continuous flow	Continuous flow/pipeline
Key idea	Sprint	TDD/pair programming	WIP limits	Automation & feedback
Visualization	Backlog / Sprint	Stories / iteration	Kanban board	Delivery pipeline
Testing	Throughout Sprint	Very strong emphasis	Throughout flow	Often highly automated

19. Most Important Exam Diagrams

Practice drawing these from memory. Keep the labels neat and use arrows consistently.

1. Layered technology

```
TOOLS
METHODS
```

```
PROCESS
QUALITY FOCUS
```

2. Software failure curves

```
Failure Rate vs Time
Hardware: infant mortality → useful life → wear-out
Software: faults removed + changes may introduce new faults
```

3. Generic process framework

```
Communication → Planning → Modeling → Construction → Deployment
                                          ↓
                                      Feedback
```

4. Waterfall

```
Requirements → Design → Implementation → Testing → Deployment → Maintenance
```

5. Prototyping

```
Requirements → Quick Design → Prototype → Evaluation → Feedback ↺
```

6. Incremental

```
Release 1 → Release 2 → Release 3 → Release 4
```

7. Spiral

```
Planning → Risk Analysis → Engineering → Evaluation → Next Cycle
```

8. Concurrent

```
Inactive → Development → Review → Revision → Baselined → Done
```

9. Unified Process

```
Inception → Elaboration → Construction → Transition
```

10. Cost of change

```
Cost of Change vs Project Progress
Traditional curve rises more steeply later; Agile practices try to make change more manageable
```

11. Scrum

```
Product Backlog → Sprint Planning → Sprint → Increment → Review → Retrospective → Next Sprint
```

12. XP

```
Planning → Design → Coding → Testing → Release
```

13. Kanban

```
TO DO → DOING → TESTING → DONE
```

14. DevOps

```
PLAN → CODE → BUILD → TEST → INTEGRATE → DEPLOY → MONITOR → PLAN
```

20. High-Priority Exam Questions

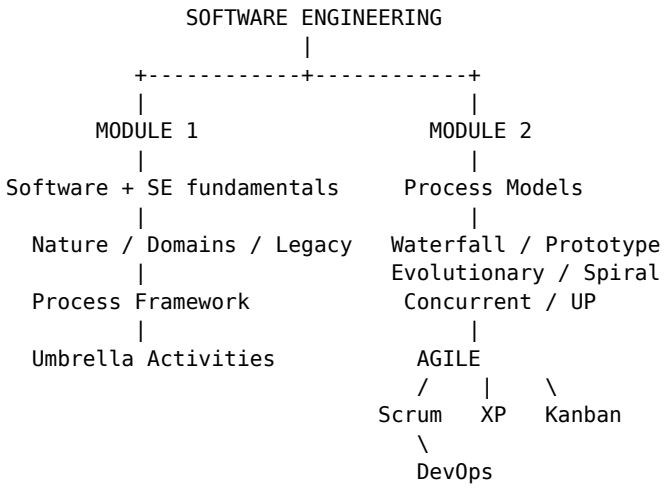
Module 1

- Explain the nature of software with suitable diagrams.
- Explain software engineering and why it is needed.
- Explain software engineering as a layered technology.
- Explain different software application domains with examples.
- What is legacy software? Explain its characteristics and problems.
- Explain the generic software process framework and process flow types.
- Explain hardware and software failure curves.
- What are umbrella activities? Explain their purpose.

Module 2

- Explain the Waterfall model with advantages and disadvantages.
- Explain the Prototyping model.
- Explain incremental and evolutionary process models.
- Explain the Spiral model and its role in risk management.
- Explain the Concurrent Development model.
- Explain the Unified Process and its four phases.
- Explain agility and the cost-of-change concept.
- Explain Agile values and principles.
- Explain Scrum: accountabilities, artifacts, events, and Sprint.
- Explain XP and practices such as TDD, pair programming, CI, and refactoring.
- Explain Kanban and WIP limits.
- Explain DevOps and the CI/CD/CD lifecycle.
- Differentiate Scrum, XP, Kanban, and DevOps.
- Differentiate iterative and incremental development.

21. Final Memory Map



One-line story: Understand software → understand software engineering → study its nature and application domains → understand legacy software → learn software processes → learn process models → understand why agility is needed → learn Scrum, XP, Kanban → understand DevOps and continuous delivery.